Panagiotis Griziotis

LT1200007

Course: Dialogue Systems & Virtual Assistants (M913)

Unibot: Building A Virtual Assistant for Students with Rasa

Description

Unibot is a demo version of a task-oriented dialogue system that helps Greek speaking college students with requests related to a course of their interest. It is designed to fulfill different requests based on the type of information a student desires to retrieve about a specific course of their semester. The system can complete three different main tasks: offer information about the detailed schedule of a course's class (day, time, professor, classroom), offer a summary of a course's subject and carry through with both these tasks at once.

In regards with the information retrieval, a database was created and connected to the system's actions consisting of information collected from the web. The selected source is the official website of the Department of Informatics and Telecommunication of the National Kapodistrian University of Athens[1]. The range of data was restricted to the undergraduate curriculum and the academic year of 2019-2020 schedule.

Architecture

The development of Unibot was completed using the open-source framework of Rasa. Following the dialogue-state architecture, our assistant consists of a sequence of components (NLU, Dialogue Tracker, Dialogue Policy, NLG) and dialogue acts, namely user intents and system actions. The intents are represented by various possible utterances a user can produce and the actions are the responsive steps that the system must predict and implement given the preceded intents.

Unibot is designed to successfully reach a custom action ***supply_answer_to_user*** to fulfill the user's initial request. When this action is predicted, the system connects to the database module, collects information from the web related to the user's request and returns it as a response message.

To achieve this goal, the assistant needs to keep in memory required information provided from the user during the conversation. To handle this business logic, a single form ***course_form*** is used which consists of consecutive template messages (e.g., "Ποιο είναι το όνομα του μαθήματος;") that prompt the user to provide necessary information

[1] https://www.di.uoa.gr.

for the assistant. This information is then stored in three different *slots*: a) request type b) semester number c) course name.

The ***request_type*** slot is filled with one of the following unique values each connected to the specific intent type that triggered the form: ***description, schedule, general***. The ***num_semester*** and ***course_name*** slots are filled with values extracted from user's inputs and are mapped to their respective *entities*. An example of a fully filled ***course_form*** is the following***:***

a) request_type: {schedule}, num_semester: {1^{o}}, course_name: {Γραμμική Άλγεβρα}

When an extracted value fills a slot, a custom action ***validate_course_form*** examines whether it is acceptable. If the slot value is successfully mapped to a value from the database, it gets replaced by the latter (e.g., extracted 'Δίκτυα Αισθητήρων' is replaced with 'Ασύρματα Δίκτυα Αισθητήρων'), otherwise the slot remains unfilled, and the system helps the user by providing him a list of allowed values (semester numbers or course names for given semester). To match an extracted value with the most similar database value for the ***course_name*** slot, levensthein distance is computed.

In case the user can't recall the information asked, while the form is active, a separate custom ***help_user_action*** is triggered to provide a list of allowed values as a response message. This expected interruption of ***course_form*** is included in an unhappy path rule and in stories.

Components

A) NLU

For our assistant to convert user messages into structured intents and entities, an NLU model is trained. Note that in this first demo version of Unibot, training data examples are limited in range and not collected by real users.

1. Tokenizers & Featurizers

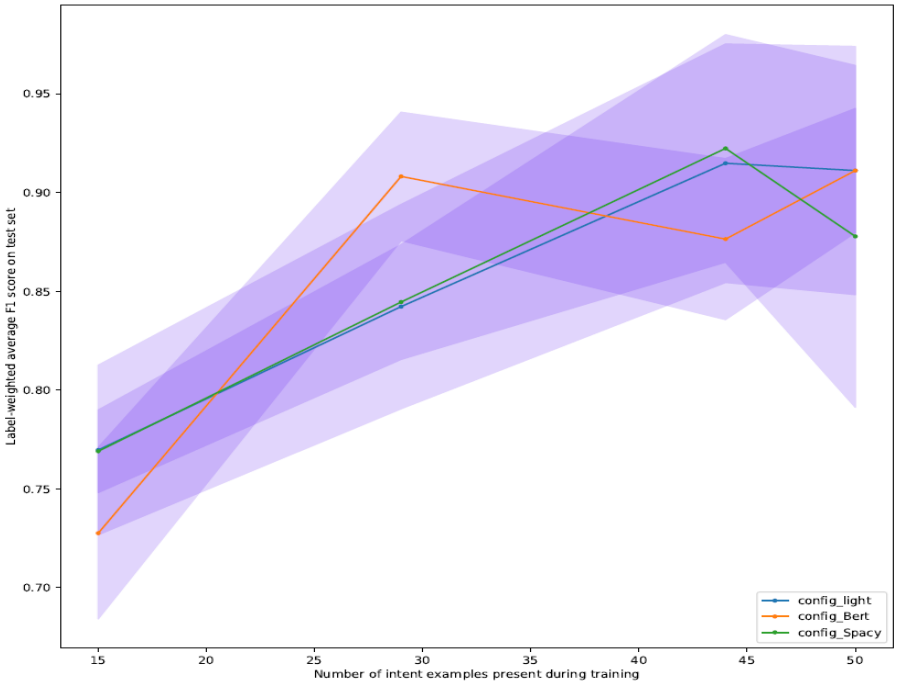
The first steps of the NLU pipeline are tokenizing user's utterance and converting tokens into numeric features. Rasa supports different components to carry through these processes. Three different configuration files were compared after training and evaluating on 80% train-20% split of the training data for the task of tuning our model.

A simple configuration is selecting a `WhiteSpaceTokenizer` for the model to create tokens based on whitespace separation of words followed by `LexicalSyntacticFeaturizer` to get sparse features by considering the syntactic position or morphology of each word in a sentence. Except for intent recognition, `LexicalSyntacticFeaturizer` usually enhances entity

recognition when its configurations are specified. Regarding our system, although ***num_semester*** entity values follow a standard pattern, that is not the case with ***course_name*** entity and therefore we cannot explicitly include an optimal morphosyntactic pattern in featurization process and change the default configuration.

Rasa also supports components that provide pre-trained word embeddings for encoding user's input into dense features. For our case, there are available pre-trained language models in Greek language that can provide pre-trained word embeddings for our data. The first model included in one of the candidate pipelines is *GREEK-BERT*[2] available in *Hugging Face's Tranformers* repository[3]. The model is loaded and initialized by the `HFTransformersNLP` component and applied to our data by `LanguageModelTokenizer` and `LanguageModelFeaturizer` which tokenize the input and create dense features respectively. The second pre-trained model tested is the Greek language model provided by spaCy[4] loaded by `SpacyNLP` and applied using the `SpacyTokenizer` and `SpacyFeaturizer` components.

After the initial tokenization and featurization, input data passes from two additional `CountVectorsFeaturizer` components to create sparse features. The `DIETclassifier` is then followed (see section 2) to make predictions and a `FallbackClassifier` to handle low confidence predictions.



[2] https://arxiv.org/abs/2008.12014
[3] https://huggingface.co/nlpaueb/bert-base-greek-uncased-v1
[4] https://spacy.io/models/el

Figure 1: *Mean and standard deviations of f1-scores across all runs for compared configuration files*

As shown in Figure 1, the results of the comparison showed that after the inclusion of the full data in training process, spaCy configurations led to a worse model than the other embedding components, regardless reaching the highest f1 score with less training data. The predictions of the model containing only sparse features (config_light) follow the most constant rate of positive change in relation to the number of training data. However, given that `SpacyEntityExtractor` is going to be added to our pipelines as a second extractor (see section 2), spaCy's dense features seem the most suitable for our NLU model.

2. Entity Extraction & DIETClassifier

After converting the user's input into word and sentence vectors, the NLU model is trained to classify it as an intent and recognize any existing entities. For both these tasks the `DIETClassifier` was trained on the training data.

The defined hyperparameters are the following:

epochs: 50
num_transformer_layers: 0
batch_size: [32, 64]
embedding_dimension: 30
weight_sparcity: 0.85
hidden_layer_sizes:
text: [256, 128]

The transformers layer was pruned due to unnecessary high computational expense given the low number of training data. Embedding dimension and weight sparcity were slightly increased.

To improve entity extraction, `SpacyEntityExtractor` was added to the pipeline. SpaCy provides pre-defined entities in Greek. Although, there is not a separate entity category for course names, talking to the bot showed that `SpacyEntityExtractor` helped to recognize a greater number of ***course_name*** entities.

B) DIALOGUE POLICY

Policy configuration is crucial for the assistant to decide which action to take at each step of the conversation. The selection of `RulePolicy` is inevitable in our case as it is a prerequisite for including the ***course_form*** in our bot. This rule-based policy is also efficient on predicting the fixed parts of conversations (e.g., greetings exchange) and

fallbacks. In rules.yml file, all necessary rules are defined, including the conditions in which ***course_form*** is activated, deactivated, or interrupted by the fixed unhappy path intent ***dont_know*** (e.g., “Δεν θυμάμαι”).

To test the efficiency of the remaining Policies, apart from `UnexpecTEDIntentPolicy`, three different dialogue models were trained and tested on test stories. There were no differences spotted between `MemoizationPolicy` and `AugmentedMemoizationPolicy` on the result metrics. In contrary, `TEDPolicy` led to better predictions achieving an average F1 score of 0.95 compared to 0.88 achieved from the other two models.

The main advantage of `TEDPolicy` is the ability to look back multiple steps and add context information for predicting the next step of the conversation. This asset is deflected in the confusion matrices of our evaluated models. Models trained using the other two types of Policies wrongly predicted the ***default_fallback_action*** instead of the ***course_form*** action (Figure 3). That is because ***course_form*** failed to continue being active until all slots are filled and at the same time recover after an interruption. With `TEDPolicy`, the confidence of predicting ***course_form*** action is increased as the dense vectors of previous predicted actions and intents are considered while predicting (Figure 2). In addition, `MemoizationPolicy` and `AugmentedMemoizationPolicy` can’t easily generalize to unseen test conversations that deviate from training examples since they only memorize training stories. Therefore, the model containing `TEDPolicy` has been selected as our final trained dialogue model. Max history has been set to 10 and training epochs to 50.

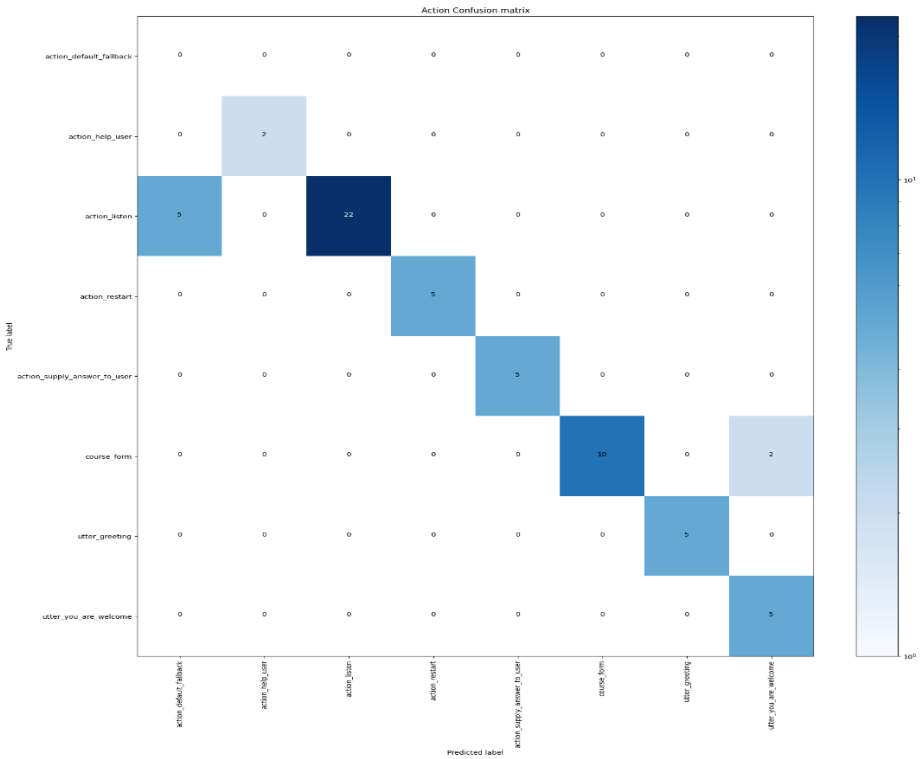


Figure 2: *Confusion Matrix for TEDPolicy model predictions (Final Model)*

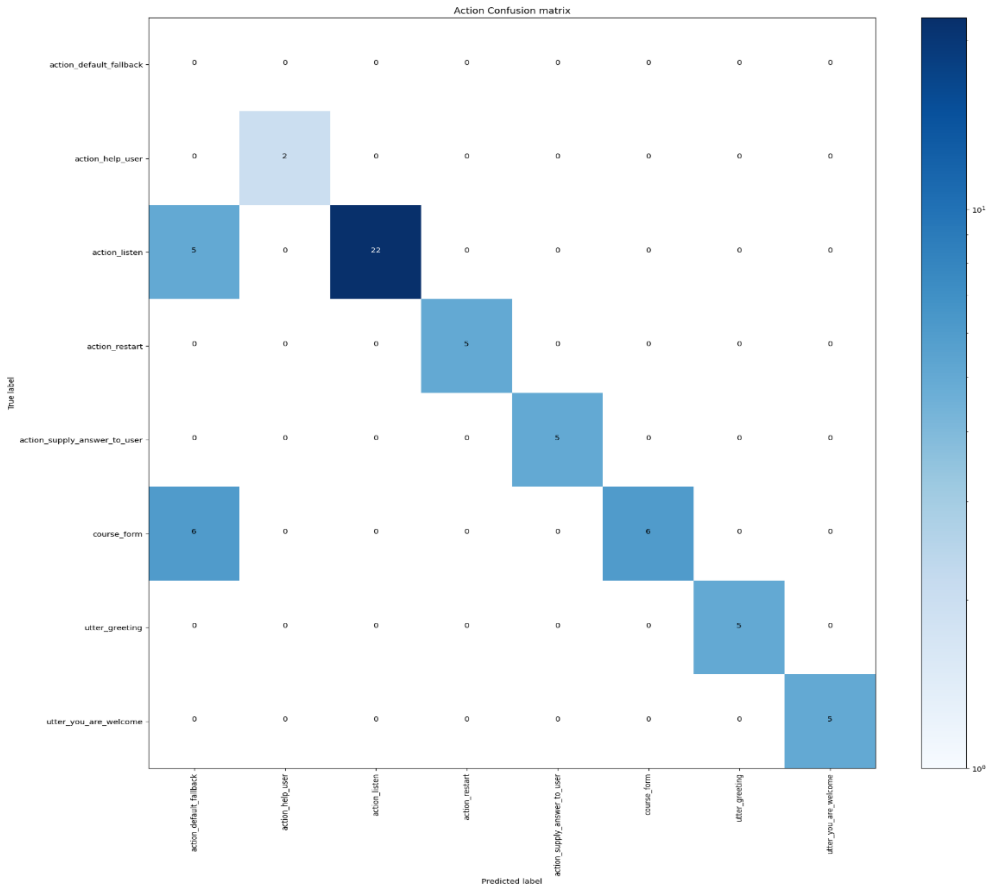


Figure 3: *Confusion Matrix for MemoizationPolicy and AugmentedMemoizationPolicy model predictions*

Evaluation Results

In terms of a component-wise evaluation, the final NLU model of our system had a bad performance in entity extraction. The results on evaluation conducted using the full number of training data showed that almost half of the entities were wrongly predicted with low confidence (Figure 4). A further analysis on the confusion matrix of the `DIETClassifier` in regards with entity recognition shows that most of the entities predicted belong in the ***no_entity*** category (Figure 5). However, as we already mentioned,while talking to the bot, ***course_name*** entity extraction seemed to improve when using `SpacyEntityExtractor` as an additional entity extractor.

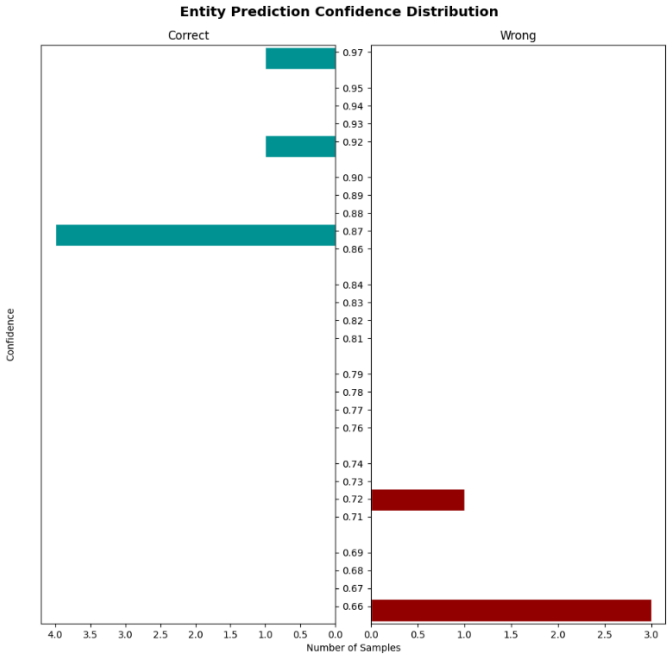


Figure 4: *DIETClassifer's entity prediction confidence distribution on the training data*

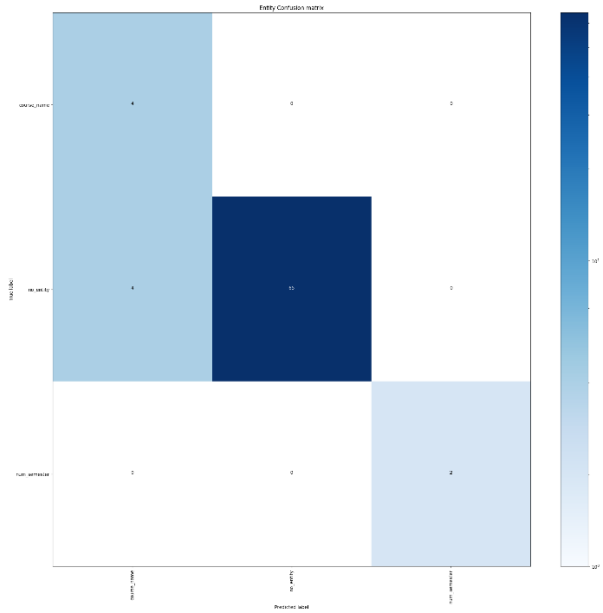


Figure 5: *DIETClassifier's confusion matrix for entity recognition*

Further evidence for low entity prediction confidence is given by the evaluation of the final dialogue model on the test stories. The results (see confusion matrix of action predictions in Figure 2 and *failed_test_stories.yml* file) show that the trained model often predicts ***default_fallback_action*** after user provides answers to questions during ***course_form.*** The model doesn't recognize any entity and instead of predicting ***action_listen*** as the next action, asks for clarifications.

Average system-wise evaluation metrics on the test stories showed a better performance for our trained dialogue model (Figure 6). However, we can't claim that these results are representative of our bot's efficiency in real conversations. Despite the fact that most of the target actions were correctly predicted, some actions carry more importance than others, for example actions inside a form.

Precision	Recall	F1-score	Support
0.97	0.88	0.92	61

Figure 6: *Weighted average evaluation scores on action prediction*

Difficulties and Future Work

This report presents the basic architecture and the evaluation of a demo version of Unibot. The range of training data as well as the potentials of the current architecture are limited. To enhance the bot's efficiency and extend its abilities in the future, obstacles must be tackled.

The increase and enrichment of the training data would be a crucial step towards our system's improvement. Course names vary on length and morphosyntactic structure and is difficult to capture when the NLU model is trained on few examples. Additionally, ordinal numbers for semester entities are currently limited to template numeric values (1ο, 2ο etc.) of predefined range.

Another main problem is the confusion between request intents. These intents share words ordered in almost the same fashion (e.g, "*θέλω να μάθω την περιγραφή ενός μαθήματος", "θέλω να μάθω για τις ώρες ενός μαθήματος"*) resulting in confusion and wrong predictions from the `DIETclassifier`.

These are few examples of problems that can be tackled in the future by deploying a conversation-driven development. This practice would allow us to get feedback from test users and improve training data as well as get new ideas for adding new features. These features could be a multi-intent recognition on a single utterance or even new forms that carry through additional requests related to a course (e.g., exams schedule) or provide university services in general (e.g., extract contact info, useful announcements etc.). Human evaluation can bring to light new problems during user and bot interactions resulting in changes of the existing conversation patterns.